

COS30002 AI for Games – D Level Custom Project Document

Written by Edward Herrod

104541862

Github Repository link: <https://github.com/104541862/COS30002-Labs/tree/main/CustomProject>

Video demonstration link: <https://youtu.be/0RiPuXNUrGc>

Table of Contents

Section 1 – Feature Summary	2
AI design	3
Behaviour tree architecture	3
Steering behaviour	3
Pathfinding algorithm	3
Justification of Changes to AI Design Plan	4
Example grey tank behaviour trees	5
Example yellow tank behaviour trees	6
Additional features	6
Debugging features	8
Section 2 – ULO Alignment	8
Section 3 – Technical Problem-Solving	9
Enemy agents shooting each other	9
Performance issues	9
Non-tank-like movement	9

Section 1 – Feature Summary

The game itself features 9 different types of AI tank agents which behave in different ways. They were laid out in the planning document as follows:

- Brown tanks are dummies – they have no movement logic and shoot bullets sporadically in a pseudo-random direction.
- Grey tanks are grunts – they move and shoot bullets sporadically in pseudo-random directions.
- Teal tanks are basic rocketeers, maintaining distance from the player if possible and shooting rockets towards the player.
- Red tanks are aggressors and will rush the player on sight, barraging them with bullets as they do so.
- Yellow tanks are the front-liners. They can lay mines and will try to take space but maintain a set distance (though smaller than the Teal tanks') from the player whilst doing so.
- Green tanks are snipers – stationary, they will use more complex trajectory calculations, including off walls, to get a rocket shot on any player, even ones they've noticed have gone behind cover (they also blend in with the background because it's green in this game!).
- Purple tanks are defenders, identifying hiding/cover spots and staying near them to force the player to their location.
- **Black tanks are elites and will use cover as leverage to peek corners and try to survive without getting overly ambitious. They're fast, attempting to dodge the players' bullets, and fire rockets directly at them. They can also lay mines. Scary stuff.**

Many of these descriptions have been upheld in the current state of the custom project build, though others are more generic. In the current game state, enemy tanks have two overarching permissions prescribed by their behaviour trees [A]:

- Moving: granted to all enemy tanks except brown and green tanks.
- Placing mines: granted to yellow and black tanks.

This results in similar behaviours across many of the enemy agents, which is intended to a degree but will be further refined later to increase the individuality of certain enemies (for instance, purple tanks don't act defensively enough).

The overall goal of the design is to support modular AI decision making, scalable enemy archetypes, real-time tactical reasoning and physically grounded movement with combat behaviour.

AI design

The AI architecture follows a relatively layered model:

- The dual behaviour tree layer defines decision making such as combat and movement.
- The steering behaviour layer defines the movement forces for separation and pathing.
- The pathfinding layer defines the high-level navigation goals and is done using A*.
- The world simulation layer defines physics, collisions and projectiles.

Behaviour tree architecture

Behaviour trees are used within this simulation to make modular decisions, reuse blocks of logic and maintain a hierarchical control flow. A significant benefit to this is that they are a much more composable alternative to finite state machines.

They are defined using nodes as follows:

- Selector nodes execute their first successful child node.
- Sequence nodes execute child nodes in order until failure.
- Action nodes execute an action.
- Condition nodes evaluate Boolean states.
- Decorator nodes enable varied and pseudorandom behaviour.

Steering behaviour

Steering behaviours govern continuous movement whilst following a path. The system is based on forces and vector normalisation, following this equation:

$$\text{desired_heading} = \text{path_direction} + \text{wall_avoidance} + \text{hole_avoidance} + \text{separation} + \text{mine_avoidance}$$

Each force is also weighted before it is applied, and are defined as follows:

- Path following is the direction of the A* route.
- Wall avoidance prevents collision with walls.
- Hole avoidance prevents collision with holes.
- Mine avoidance prevents the agent from blowing itself up.
- Separation prevents agent clustering.

Pathfinding algorithm

The world is split into a uniform square grid space, made up of nodes which define their walkability, coordinates and neighbour information. A* is used to compute the optimal path from one point to another.

Justification of Changes to AI Design Plan

One of the core design principles governing the development of this simulation is modularity, with focus also being placed upon composability and scalability. The original planning document (task 18) for this simulation envisaged the use of hierarchical finite state machines (HSMs) as were used for previous tasks; however, to support these core design principles, the decision was made to switch from a HSM architecture to a split behaviour tree architecture. It was quickly realised during development that the HSM approach would have resulted in duplication of transitions, deeply nested state hierarchies and brittle coupling between movement and combat logic.

Behaviour trees can address these limitations by dissolving the AI into reusable decision nodes which can be composed in a hierarchical fashion without enforcing strict exclusive global states. This allows for the independent evaluation of movement and combat, which is extremely similar to the way tanks operate, realistically speaking.

Example grey tank behaviour trees

```

GREY_MOVEMENT_TREE = Selector([
    Sequence([
        Condition(player_too_close),

        Action(lambda a, d: setattr(
            a,
            "path_timer",
            max(0.0, a.path_timer - d)
        )),

        Condition(lambda a: a.path_timer <= 0),

        Action(lambda a, d: setattr(a, "path", grey_path(a))),
        Action(lambda a, d: setattr(a, "path_timer", 2.0)),
    ]),

    Sequence([
        Condition(player_too_close),
        Action(lambda a, d: a.follow_path(a.path, d))
    ])
])

GREY_COMBAT_TREE = Selector([
    Sequence([
        HasLineOfSight(lambda a: a.world.player),

        AimAt(
            lambda a: a.world.player,
            smooth=0.25 # reduce jitter
        ),

        CanShoot(),

        RandomChance(0.08), # slightly more natural pacing
        Shoot()
    ]),

    # fallback idle aiming
    Action(lambda a, d: a.update_turret_sway(d))
])

```

If the player is too close:

- Move to a position which is a set distance away from the player.

If the grey tank has line of sight to the player:

- Aim roughly towards the player

If the grey tank can shoot:

- Shoot at random intervals

Example yellow tank behaviour trees

```

YELLOW_MOVEMENT_TREE = Selector([
    Sequence([
        RandomChance(0.03),
        Action(lambda a, d: setattr(a, "path", grey_path(a))),
        Action(lambda a, d: setattr(a, "path_timer", 2.0)),
    ]),

    Sequence([
        Action(lambda a, d: a.follow_path(a.path, d))
    ])
])

YELLOW_COMBAT_TREE = Selector([
    Sequence([
        CanPlaceMine(),
        RandomChance(0.001), # occasional trap laying
        PlaceMine()
    ]),

    Sequence([
        HasLineOfSight(lambda a: a.world.player),

        AimAt(
            lambda a: a.world.player,
            smooth=0.2,
            noise=0.05
        ),

        CanShoot(),
        RandomChance(0.2),
        Shoot()
    ]),

    Action(lambda a, d: a.update_turret_sway(d))
])

```

- Move to a position which is a set distance away from the player.

If the yellow tank can lay a mine:

- Lay a mine at random intervals

If the yellow tank has line of sight to the player:

- Aim roughly towards the player

If the yellow tank can shoot:

- Shoot at random intervals

Additional features

To prevent enemies from shooting each other, a hard buffer around enemy agents has been added that they cannot shoot towards, as a rule (i.e. their predicted shot ray cast intersects with the buffer zone).

To make the shooting for certain tanks (namely the green tank) more accurate, there has been an attempt to have these agents account for bullets bouncing off walls whilst they aim. This has yet to be fully implemented, but a function for it does exist:

```
def find_bounce_shot(self, samples=60, max_bounces=1):
    from math import cos, sin, radians
    import random

    origin = self.get_muzzle_position()
    player = self.world.player

    best_dir = None
    best_score = 0.0

    for i in range(samples):
        angle = random.uniform(0, 360)
        direction = Vector2D(cos(radians(angle)), sin(radians(angle)))

        path = self.simulate_bullet_path(origin, direction, max_bounces=max_bounces)

        hit_player = False
        score = 0.0

        for start, end in path:
            seg = end - start
            seg_len = seg.length()
            if seg_len < 1:
                continue

            seg_dir = seg.get_normalised()

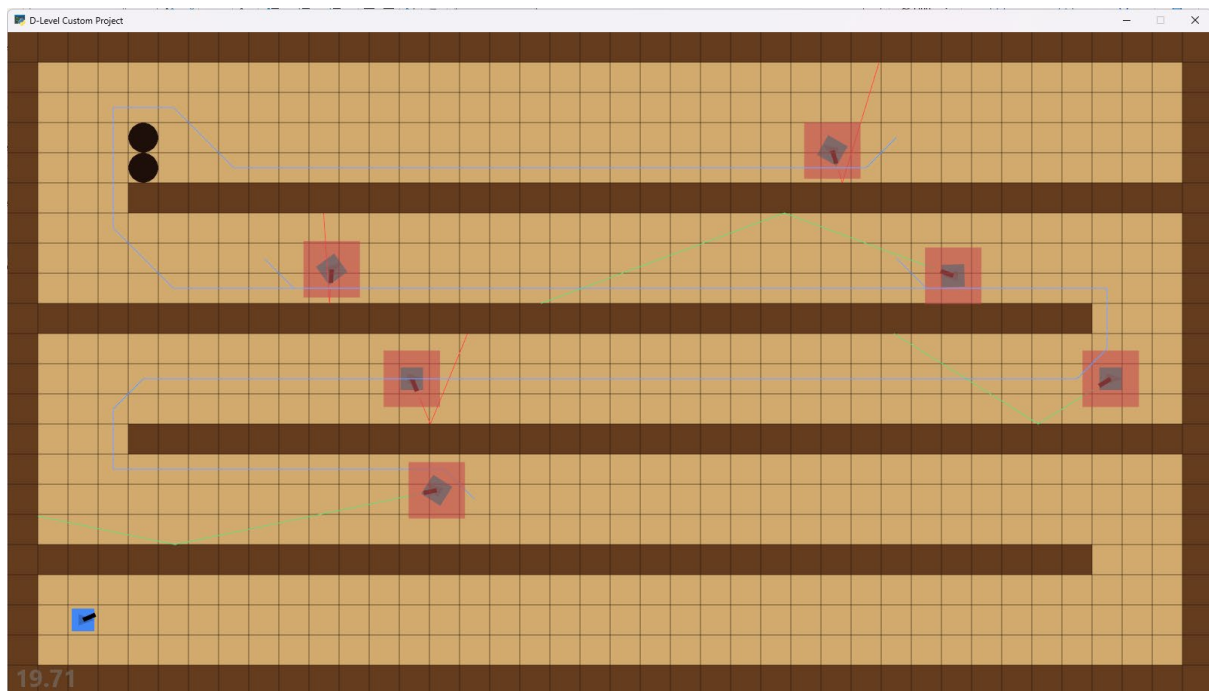
            ax, ay, aw, ah = player.get_aabb()
            hit, t = _ray_aabb_t(start, seg_dir, ax, ay, aw, ah)

            if hit and 0 <= t <= seg_len:
                hit_player = True
                score = 1.0 / max(1.0, (start - player.pos).length())
                break

        if hit_player and score > best_score:
            best_score = score
            best_dir = direction

    return best_dir
```

Debugging features



By hitting the “I” key, users are shown a debugging tool which displays the following features:

- Shooting telemetry which highlights red if the agent cannot shoot
- Grid squares
- A* pathfinding paths
- Red no-shoot buffer zones
- The current frame rate

Section 2 – ULO Alignment

The following table has been created to describe the ULO alignment of implemented features in the current state of the simulation:

Feature	ULO 1	ULO 2	ULO 3	ULO 4	ULO 5
Behaviour tree architecture	X				
A* tactical pathfinding		X			
Separation steering			X		
Agent marksmanship				X	
Combined behaviour trees and resultant AI behaviour					X

Section 3 – Technical Problem-Solving

Several problems were encountered whilst designing the code base for this AI simulation. Many of these were to do with low-level behaviour not lining up with the envisaged design of AI agents.

Enemy agents shooting each other

Whilst designing the brown tanks, it was discovered that they would shoot one another if a logical barrier wasn't put in place to prevent them from doing so. This was unrealistic, so a buffer zone was added around all enemy tanks to prevent them from shooting each other and is accessible to view via the debugging screen. To extend this, bullet avoidance could be added to moving agents to prevent them from running into bullets.

Performance issues

This issue is still encountered to a degree but is not as severe as it was before changes were implemented to rectify it. When many AI agents are present on the screen at once, the persistent pathfinding and bullet trajectory prediction casting would cause measurable performance issues (FPS < 10). To solve this, the number of times the AI attempts to path find to a new tactical location was reduced.

Non-tank-like movement

During the initial development phase of this simulation, movement was far too simplistic and did not reflect the design philosophy of a tanks game, with agents sliding around without needing to turn. This was altered to have a triangle rendered above the main chassis showing the direction of forward movement and for the agent to need to turn before being able to move in the desired direction.